

AI-Assisted Project Delivery & UAT Management

A fictionalized digital municipal platform case study

Role	Project Manager — Project Delivery & UAT Coordination
Focus	Project planning · Requirements · UAT · RAID · Stakeholder coordination
AI Layer	Meeting assistant · Change impact analysis · Risk analysis
Tools	Claude · Excel / Google Sheets · Jira · GitHub

Portfolio Objective

Demonstrate how a Project Manager can combine structured delivery practices with AI-assisted workflows to turn client discussions into validated requirements, actionable work, UAT coverage, and release-readiness decisions.

***Note:** This case study is fictionalized for portfolio use. The client, platform, requirements, and change requests are invented to demonstrate the delivery process and AI-assisted workflow pattern — no confidential client information is included.*

Client Discussion	AI Analysis	PM Validation	Jira	Development	QA / UAT	Closure
-------------------	-------------	---------------	------	-------------	----------	---------

3 AI Workflows Meeting assistant, change impact analysis, and risk analysis — each with a defined validation step.	22 Tracked Items Across bugs, change requests, and post-go-live follow-ups, with a formula-driven dashboard.	Human-in-the-Loop Every AI output is validated by the PM before it becomes a ticket, decision, or commitment.
--	--	---

1. Project Context & My Role

This case study is based on a fictionalized digital municipal payments platform implementation ("CivicPay") for a fictional client, Northbridge Civic Solutions. The engagement ran for approximately three months, covering resident-facing bill payment and a municipal staff admin console. I was the sole Project Manager on the engagement, responsible for delivery coordination end-to-end.

My Responsibilities

Area	Contribution
Requirements	Prepared and updated the BRD; translated client discussions into structured requirements and change requests.
Client coordination	Led client calls, captured clarifications, prepared MOMs, tracked client actions.
Development coordination	Followed ticket progress, coordinated clarifications, tracked pending fixes.
UAT	Maintained the UAT tracker, coordinated test execution, raised defects, followed fixes through retest.
Project monitoring	Reviewed open / closed / overdue tickets, blockers, UAT progress, and upcoming releases.
Delivery governance	Maintained RAID items, tracked dependencies, supported release / go-live readiness.

The Operating Problem

Project information was distributed across client discussions, tickets, UAT observations, dependencies, and status updates. The challenge was not simply recording information — it was turning that information into decisions, follow-ups, and release-ready actions without losing business context.

The Solution

I structured delivery around a repeatable loop: capture → analyze → validate → execute → verify. AI was introduced at the analysis layer to accelerate requirement interpretation, meeting extraction, and project-health summaries — while human validation remained mandatory before any decision or development commitment.

1. Capture	2. Analyze	3. Validate	4. Execute	5. Verify
------------	------------	-------------	------------	-----------

2. AI Workflows Built for Project Delivery

The AI layer was designed as a copilot. Its job is to structure information and surface questions — not to make business decisions or silently invent missing requirements.

Workflow A — Change Impact Analysis

Input	AI Analysis	Human Validation	Output
Client change request	Impacted modules, risks, dependencies, regression areas, missing business rules	PM validates business rules and assumptions with client	Validated requirement + Jira + UAT cases

Example: a request to replace a resident "Delete Account" action with a staff-initiated "Merge Duplicate Accounts" flow. AI surfaced impacts across the data model, payment-history reconciliation, staff permissions, and reporting. Two unknowns were then explicitly validated with the client: whether autopay survives a merge, and whether the resident is notified by email afterward.

Workflow B — AI Meeting Assistant

Input	AI Analysis	Human Validation	Output
Client / UAT meeting notes	Decisions, actions, owners, deadlines, dependencies, Jira candidates	Verify owners/deadlines were actually stated and decisions confirmed	Action tracker + Jira/UAT follow-ups

Workflow C — AI Risk Analysis

Input	AI Analysis	Human Validation	Output
Requirements, dependencies, open change requests	Candidate risks with rationale	PM assigns impact, owner, mitigation	Logged as active risk in RAID

AI governance principle: AI identifies → PM verifies → Client decides where required → Team executes. This prevents AI-generated assumptions from becoming uncontrolled project scope.

3. Requirements & Change Impact Analysis

Two representative change requests demonstrate how the AI workflow converts ambiguous client language into development- and UAT-ready requirements.

Change Request 1 — Merge Duplicate Resident Accounts

Validated rule	Final behavior
Authority	Only municipal staff (not residents) can initiate a merge.
Account matching	Staff select two accounts confirmed to belong to the same resident.
Payment history	Transaction history is reconciled under a single surviving account ID.
Autopay	Any active autopay setup on either account survives the merge without interruption.
Notification	Resident receives an email confirmation after a merge completes.
Audit	Merge action is recorded in the audit trail with both source account IDs.

Impact areas: Account Data Model · Payment History · Staff Permissions · Notifications · Audit Trail · UAT / Regression

Change Request 2 — Multi-Currency Fee Display (Border Municipality Pilot)

Validated rule	Final behavior
Scope	Applies only to a fixed pilot list of border municipalities, not a general config flag.
Display	Transaction fees shown in local currency plus a secondary currency for resident clarity.
Conversion rate	Updated manually on a weekly cadence — not a live/real-time feed.
Reporting	Municipal admin reports clarify which currency totals are expressed in.
Regression	Non-pilot municipalities are unaffected by the secondary display.

Impact areas: Fee Display UI · Reporting · Regression (non-pilot municipalities) · UAT

Why This Matters

The two changes were deliberately treated differently. Merging accounts changes business behavior and data lifecycle; currency display changes presentation and reporting only. The AI analysis helps surface that distinction before tickets and UAT scenarios are finalized — and in both cases, the AI's first draft made an assumption (a general-purpose feature flag, a live conversion feed) that the PM corrected against what the client actually agreed to.

4. Risk, Issue, Dependency & UAT Management

A lightweight RAID structure converted day-to-day project observations into trackable delivery actions.

Type	Representative item	PM response
Risk	Payment gateway sandbox availability could delay end-to-end testing.	Escalated to vendor for a dedicated test window.
Risk	Municipal staff have limited UAT availability during tax season.	Scheduled short UAT blocks around peak hours.
Issue	Report export timing out on large date ranges.	Raised ticket, tracking fix and retest.
Issue	Duplicate resident accounts possible under an edge case.	Raised, tracked through to retest.
Dependency	Legacy billing data export needed for account matching.	Tracked with migration vendor.
Dependency	Final branding assets required for resident portal.	Followed up with client marketing team.

UAT Management Approach

Stage	Activity
Prepare	Map requirements to test scenarios and identify impacted modules.
Execute	Coordinate testing, record defects, update UAT tracker.
Coordinate	Follow development fixes, client actions, and deployment status.
Retest	Validate fixes and perform regression where changes affect existing behavior.
Readiness	Review open high-priority items, blockers, and pending dependencies before release.

Representative UAT Coverage — Merge Duplicate Accounts

Test area	Expected result
Staff-initiated merge	Two accounts combine into a single active account.
Unauthorized user	A resident cannot self-initiate a merge.
Payment history	Full transaction history visible under the surviving account.
Active autopay	Autopay continues without interruption post-merge.
Notification	Resident receives a merge-confirmation email.
Audit trail	Merge action recorded with both original account IDs.

5. Project Dashboard & Delivery Control

The dashboard consolidates the same information reviewed during project monitoring into a recruiter-friendly project control view.

Snapshot



AI-Generated Project-Health Interpretation

UAT progress is affected by the pending payment gateway sandbox dependency. Several high-priority and overdue tickets require follow-up with development. Both change requests need development and UAT validation. The PM response is to verify the underlying tracker, prioritize actions, follow dependencies, and assess release readiness before committing to a go-live date.

Priority Action Model

Priority	Action	Owner / collaboration
High	Resolve payment gateway sandbox dependency	Client + PM
High	Confirm timelines for overdue development tickets	Development + PM
High	Validate change-request impact	PM + Development + QA
Medium	Complete regression testing	QA + PM
High	Review release / go-live readiness	Project team + Client

What the Dashboard Demonstrates

Rather than a static status report, the dashboard connects operational data to project actions. All KPI values are formula-driven from the underlying ticket log — not hardcoded — so the summary stays consistent with the data whenever the tracker is updated.

6. Outcome, Skills Demonstrated & Portfolio Takeaway

This case study is designed to demonstrate a practical Project Manager operating model — not to claim that AI independently managed a production project.

Capabilities Demonstrated

Capability	Evidence in this case study
Requirements management	BRD preparation, requirement clarification, validated change requests.
Stakeholder management	Client calls, MOMs, clarifications, coordination with development.
Project tracking	Open / closed / overdue tickets, blockers, UAT progress, release activities.
UAT management	Test coordination, defect tracking, retesting, regression coverage.
RAID management	Risks, assumptions, issues, and dependencies tracked with owners and actions.
Change management	Impact analysis and structured conversion of requests into Jira/UAT work.
AI-assisted delivery	Three repeatable AI workflows with human validation at the decision point.

Key Learning

The strongest use of AI in project delivery is not simply generating documents faster. It is using AI to make hidden project information easier to see: missing business rules, downstream impact, unresolved actions, dependencies, and emerging delivery risks. The PM still owns validation, prioritization, and stakeholder decisions.

Portfolio Artifacts Created

Artifact	Purpose
Project Charter / Plan	Define scope, stakeholders, timeline, delivery approach.
RAID Log	Track risks, assumptions, issues, dependencies.
AI Change Impact Analysis	Analyze client changes and surface missing rules.
AI Meeting Assistant	Convert meeting notes into decisions and actions.
AI Risk Analysis	Surface candidate risks feeding into the RAID log.
Project Dashboard	Consolidate project data into an action-oriented, formula-driven status view.

Recruiter takeaway: Project Manager with hands-on experience across requirements, stakeholder coordination, UAT, defect tracking, and delivery monitoring — with a portfolio demonstrating practical AI integration into project-management workflows.

Public-portfolio note: all examples in this case study are fictionalized and intended to demonstrate process, problem-solving, and tool usage without exposing any confidential client information.